

MIPS

Instruction Set Architecture

Module.5

CDA 3103C

Mesut Ozdag, Ph.D.
mesut.ozdag@ucf.edu

Introduction

- Hardware
 - Voltages
 - Logic Gates
 - Latches
 - Flip-Flops
 - Registers
 - Adders
 - Arithmetic Logic Unit

Introduction

- Hardware Languages
 - Instruction Sets
 - MIPS Assembly Language
 - Machine Code
 - Binary (e.g., 1001 1110 0000 1010)
 - Hexadecimal (e.g., 9 14 0 10 = 9E0A)

Instruction Set

- Instructions: the words of a computer language
- Instruction set: vocabulary
 - Repertoire of instructions of a computer
- Instruction sets may differ from computer to computer, but have many things in common.
 - Computational operations
 - Memory access & addressing
 - Branches
 - Procedure calls

MIPS Instruction Set

- All instruction sets promote a common goal.
 - Make it easy to build the hardware and the compiler while maximizing performance and minimizing cost.
- MIPS is a popular instruction set and has large share of the embedded core market.

Arithmetic Operations

R-Type Instructions

- `<operation> <destination_register>, <source_register1>, <source_register2>`

- Addition

C code:

```
a = b + c;
```

Compiled code format for a MIPS processor:

```
add    a, b, c
```

Arithmetic Operations

- Each instruction performs only one operation.
- Each instruction must have three variables.
 - Two sources and one destination.

Arithmetic Operations

- $a = b + c + d + e;$

add	a, b, c	$\#a = b + c$
add	a, a, d	$\#a = b + c + d$
add	a, a, e	$\#a = b + c + d + e$

Arithmetic Operations

- Each line can only have one instruction on it.
- Anything following a # is a comment.

Arithmetic Operations

- From C to MIPS

`a = b + c;`

`d = a - e;`

`add a, b, c`

`sub d, a, e`

Arithmetic Operations

- From C to MIPS

$f = (g + h) - (i + j);$

add	t0, g, h	#temporary variable t0 = g + h
add	t1, i, j	#temporary variable t1 = i + j
sub	f, t0, t1	#f = t0 - t1

Operands

- Instructions use register operands
- Registers
 - Made from Flip-Flops
 - Primitive used in hardware design
- Register Size
 - 32 bits
 - Called a “word”
- Number of Registers
 - 32 registers in the register file
 - Use for frequently accessed data
 - Numbered 0 to 31

Operands

- MIPS naming conventions
 - \$00 - \$31
 - \$XX
 - A \$ followed by two characters that represent the register
- Assembler names
 - \$t0, \$t1, ..., \$t9 for temporary values
 - \$s0, \$s1, ..., \$s7 for saved variables

Arithmetic Operations

- From C to MIPS

$f = (g + h) - (i + j);$

f	g	h	i	j
\$s0	\$s1	\$s2	\$s3	\$s4

Solution?

add	\$t0, \$s1, \$s2	#temporary variable t0 = g + h
add	\$t1, \$s3, \$s4	#temporary variable t1 = i + j
sub	\$s0, \$t0, \$t1	#f = t0 - t1

Multiplication & Division

- Multiply

<i>mult</i>	<i>\$t1, \$t2</i>	<i># HiLo = \$t1 * \$t2</i>
<i>mflo</i>	<i>\$t0</i>	<i># t0 = Lo</i>
<i>mfhi</i>	<i>\$t3</i>	<i># t3 = Hi</i>

```
      0011
    * 0010
    -----
0000 0110
```

```
      0011
    * 0110
    -----
0001 0010
```

- Divide

<i>div</i>	<i>\$t1, \$t2</i>	<i># Lo = \$t1 / \$t2, Hi = \$t1 % \$t2</i>
<i>mflo</i>	<i>\$t0</i>	<i># t0 = Lo</i>
<i>mfhi</i>	<i>\$t3</i>	<i># t3 = Hi</i>

Example

- Consider the following line of C code:

$a = (b+c) - (b-d) + (c/d)$

where integers $\{a, b, c, d\}$ reside in $\{\$s1, \$s2, \$s3, \$s4\}$ respectively.

- Solution?

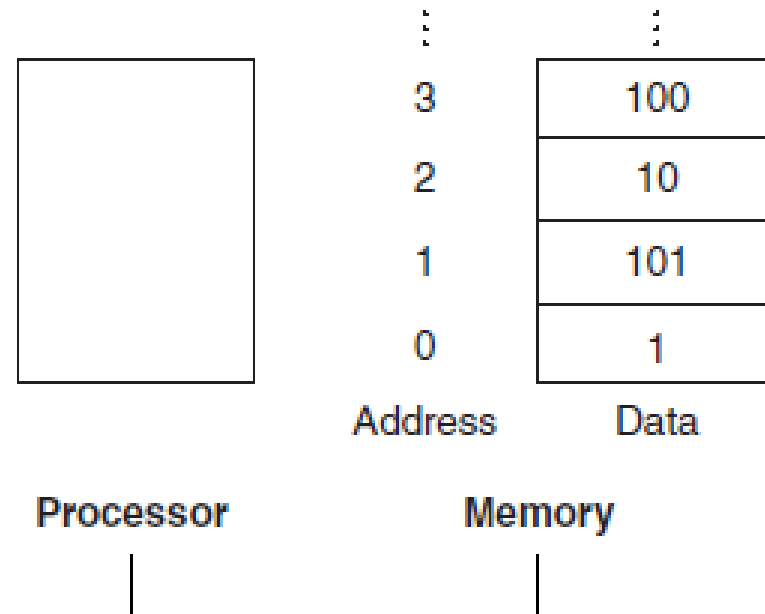
```
add    $t0, $s2, $s3
sub     $t1, $t2, $s4
div     $s3, $s4
mflo    $t2
sub     $t0, $t0, $t1
add     $s1, $t0, $t2
```


Memory Operands

- Simple variables
 - Integers, Characters, etc.
- Data Structures
 - Arrays, Structures
- Store data structures in main **memory**

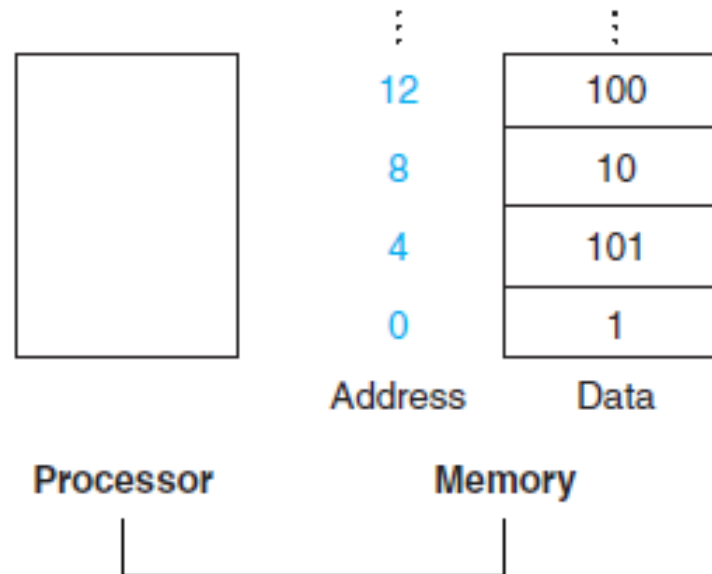
Memory Operands

- To use data from main memory:
 - Load values from memory into registers
 - Store result from register to memory



Memory Operands

- Memory is byte addressed
 - Each address identifies an 8-bit byte
- Words are aligned in memory
 - Address must be a multiple of 4



Memory Operands

- load
 - Copies data from memory to a register
- store
 - Copies data from a register to memory

Memory Operands

lw destination, constant(register)

The “load word” instruction takes the sum of the constant and the register to determine a memory address. The data at this address is placed in the destination register.

Memory Operands

- From C to MIPS
 - A is an array of 100 words
 - g is a variable in \$s1
 - h is a variable in \$s2
 - base address of A is in \$s3

`g = h + A[8];`

- First, we have to transfer A[8] to a register.
- A[8] is stored in memory address $\$s3 + 8 \times 4$

```
lw    $t0, 32($s3)  # t0 = A[8]
add   $s1, $s2, $t0  # g = h + A[8]
```

Offset
Base Register



Memory Operands

sw data, constant(register)

The “store word” instruction takes the sum of the constant and the register to determine a memory address. The data in the first register operand will be placed at this address.

Memory Operands

- From C to MIPS
 - A is an array of 100 words
 - h is a variable in \$s2
 - base address of A is in \$s3

$A[12] = h + A[8];$

```
lw    $t0, 32($s3) # t0 = A[8]
add   $t0, $s2, $t0 # t0 = h + A[8]
sw    $t0, 48($s3) # store t0 in A[12]
```


Constant Operands

- Sometimes we need to use a constant value
- So far, we would need to load the constant into a register
 - Requires two instructions: load word and add
 - *lw \$t0, AddrConstant4(\$s1) # \$t0 = constant 4*
 - *add \$s3, \$s3, \$t0 # \$s3 = \$s3 + \$t0 (\$t0 == 4)*
- Instead, the “add immediate” instruction allows us to use a constant instead of one of the register operands

Constant Operands

- From C to MIPS

`s = s + 4;`

`addi $s0, $s0, 4`